

Tutorial 2: Hack Assembler Matrix Operations

Kiran Kumar P

Part 1: Building the Assembler

1(a) APIs of Parser, Code, and SymbolTable

Parser Module

The Parser reads the input file and processes each command.

- `hasMoreCommands()` – Checks if more commands exist
- `advance()` – Moves to next command
- `commandType()` – Returns A, C, or L command
- `symbol()` – Returns symbol of A/L command
- `dest()` – Returns destination field
- `comp()` – Returns computation field
- `jump()` – Returns jump field

Code Module

Translates mnemonics into binary.

- `dest(mnemonic)`
- `comp(mnemonic)`
- `jump(mnemonic)`

SymbolTable Module

Maintains symbol-to-address mapping.

- `addEntry(symbol, address)`
- `contains(symbol)`
- `getAddress(symbol)`

1(b) Variable Memory Mapping

Custom variables start at RAM address **16**.

Part 2: 3×3 Matrix Multiplication

2(a) Memory Requirements

Each matrix contains 9 elements.

- Matrix A: 9 locations
- Matrix B: 9 locations
- Matrix C: 9 locations

Total memory required = **27 locations**

2(b) Multiplication Strategy

Using repeated addition:

$$a \times b = a + a + \cdots + a \text{ (b times)}$$

- Nested loops for row and column traversal
- Inner loop for repeated addition
- Temporary registers for intermediate results

2(c) Matrix Initialization

Static Initialization

Values are predefined in the assembly file.

```
@value  
D=A  
@address  
M=D
```

Dynamic Initialization

Values are assigned during runtime using input.

```
@KBD  
D=M  
@address  
M=D
```

Part 3: Memory Layout and Efficiency

3(a) Row-major vs Column-major

Answer: Yes, it affects efficiency.

In matrix multiplication $C = A \times B$, each element of $C(i, j)$ is computed as the dot product of the i -th row of A and the j -th column of B .

- Matrix A should be stored in **row-major order**, so that elements of a row are stored contiguously. This allows efficient sequential access when iterating across a row.
- Matrix B should be stored in **column-major order**, so that elements of a column are stored contiguously. This avoids costly address recomputation when accessing columns.

If both matrices were stored in row-major order, accessing columns of B would require non-contiguous memory jumps, increasing the number of instructions and reducing efficiency.

Thus, storing A in row-major and B in column-major order minimizes address computation overhead and improves overall execution efficiency.